Code Verification in Rust with Verus

Matt Grimes

Oct 30 2026

What is code verification

Z3

Verus

Verification as LLM Guardrails

What does it mean for code to be "correct?"

To ask if code is "correct," we have to formulate a formal claim about what the code does.

Taking a max

If *max* returns the max of two integers, x and y, we expect the following properties:

- $max(x, y) \in \{x, y\}$
- $max(x, y) \geq x$
- $max(x, y) \geq y$

Taking the max of two integers

We can state the logical claim as

$$\forall x, y, m \in \mathbb{Z},\ m = \max(x, y) \implies (m = x \vee m = y) \wedge m \geq x \wedge m \geq y$$

In general, $a \implies b$ is equivalent to $\neg a \vee b$. We can thus rewrite

$$\forall x, y, m \in \mathbb{Z},\ \neg(m = \max(x, y)) \vee ((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)$$

Equivalently, the implementation of max has a bug if

$$\exists x, y, m \in \mathbb{Z},\ (m = \max(x, y)) \wedge \neg((m = x \vee m = y) \wedge m \geq x \wedge m \geq y)$$

The general formulation

Let s and s' denote program states, and let $C(s, s')$ mean that program C can terminate in s' when started in s.

$$\forall s, s',\ P(s) \land C(s, s') \Longrightarrow Q(s') \quad := \quad \{P\}\ C\ \{Q\}$$

- P: a predicate on the initial state (the precondition).
- Q: a predicate on the final state (the postcondition).
- Initial values can be retained in the state to express input–output relations.

Applied to max

In our example we have

- P: x and y are integers
- C: the implementation of max
- Q: $m \geq x \land m \geq y \land (m = y \lor m = x)$

What about the implementation?

In pseudo-code, we can define a naive implementation of max:

```
fn max(x: int, y: int) -> (m: int)
{
    if x > y
    {
        return x;
    } else
    {
        return y;
    }

}
```

Logically, this implementation says

$$m = \max(x, y) := ((x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y))$$

Putting it all together

We can combine everything on the prior slides into the following statement: our implementation of max is *incorrect* if and only if

$$\exists x, y, m \in \mathbb{Z} : [(x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y)] \\ \wedge \neg [(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]$$

We can see by inspection that the claim is false, there is no counter-example, and the implementation is proven correct.

Z3: satisfiability relative to a theory

In the last section, we asked whether or not

$$\exists x, y, m \in \mathbb{Z} : [(x > y) \wedge (m = x)) \vee (\neg(x > y) \wedge (m = y)] \\ \wedge \neg [(m = x \vee m = y) \wedge m \geq x \wedge m \geq y]$$

is true.
This is exactly the type of statement the SMT/SAT solver Z3 is designed to answer.

Z3

SMT: satisfiability modulo theories

Given a formula and a background "theory", is the formula satisfiable?

- Boolean combinations of constraints in supported theories.
- Examples: integer arithmetic, fixed-width bit-vectors, arrays.
- The theory determines the meaning of symbols such as $+$ and $\leq$.

For our example: linear constraints over the integers.

Source: Microsoft's Z3 Guide

Prove a property by searching for its violation

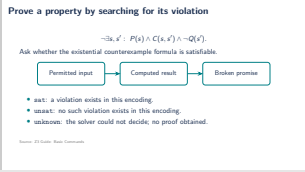
$$\neg\exists s, s' : \; P(s) \wedge C(s, s') \wedge \neg Q(s').$$

Ask whether the existential counterexample formula is satisfiable.



- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands



A timeout also leaves the obligation unproved. With abstractions, a satisfying model may require interpretation before calling it a real program counterexample. Our small exact arithmetic example has a direct interpretation.

Ask Z3 to find any bugs

```
(define-fun is_max ((m Int) (x Int) (y Int)) Bool
    (and
        m>=x
        m>=y
        (or
            m=x
            m=y
        )
    )
)
(define-fun max ((x Int) (y Int)) Int
    (ite (< x y) y x)
)

(declare-const x Int)
(declare-const y Int)
(declare-const m Int)
(assert
    (and
        (= m (max x y))
        (not (is_max m x y))
    )
)
(check-sat)
(get-model)
```

`sat`: for example, $x = 0$, $limit = 0$ gives $r = 1$.

Ask Z3 to find any bugs



aat: for example, $x = 0$, $limit = 0$ gives $r = 1$.

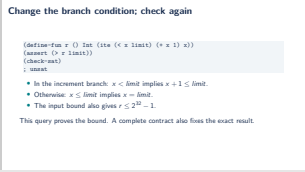
Read prefix notation aloud: (<= x limit) means x <= limit; ite means if-then-else. `QF_LIA` selects quantifier-free linear integer arithmetic. Free constants are assigned values in a satisfying model, so the query searches existentially. This is examples/counter-bug.smt2; the exact model may differ.

Change the branch condition; check again

```
(define-fun r () Int (ite (< x limit) (+ x 1) x))
(assert (> r limit))
(check-sat)
; unsat
```

- In the increment branch: $x < \mathit{limit}$ implies $x + 1 \leq \mathit{limit}$.
- Otherwise: $x \leq \mathit{limit}$ implies $x = \mathit{limit}$.
- The input bound also gives $r \leq 2^{32} - 1$.

This query proves the bound. A complete contract also fixes the exact result.



The full standalone query is in examples/counter-fixed.smt2. It uses the same input assertions as the previous slide. Verus automates generation of obligations from the source instead of requiring us to manually maintain this encoding.

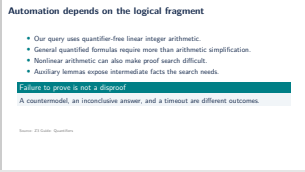
Automation depends on the logical fragment

- Our query uses quantifier-free linear integer arithmetic.
- General quantified formulas require more than arithmetic simplification.
- Nonlinear arithmetic can also make proof search difficult.
- Auxiliary lemmas expose intermediate facts the search needs.

Failure to prove is not a disproof

A countermodel, an inconclusive answer, and a timeout are different outcomes.

Source: Z3 Guide: Quantifiers



A specialist may ask about decidability. Distinguish a decidable fragment from the whole verification language and from practical resource bounds. Do not suggest that SMT is a complete decision procedure for arbitrary mathematical statements. Quantifier instantiation is generally heuristic.

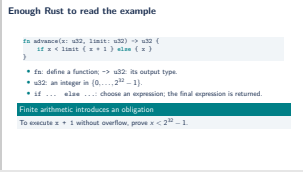
Enough Rust to read the example

```
fn advance(x: u32, limit: u32) -> u32 {
    if x < limit { x + 1 } else { x }
}
```

- `fn`: define a function; `-> u32`: its output type.
- `u32`: an integer in $\{0, \ldots, 2^{32} - 1\}$.
- `if ... else ...`: choose an expression; the final expression is returned.

Finite arithmetic introduces an obligation

To execute `x + 1` without overflow, prove $x < 2^{32} - 1$.



Rust is a systems programming language with static checks on types and memory use. We need no ownership or lifetime syntax in this talk. Ordinary u32 addition must not be treated as mathematical addition beyond its range: overflow behavior depends on settings, while explicit wrapping arithmetic is a different operation. Verus checks absence of overflow for the addition shown.

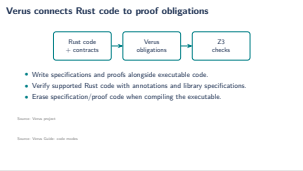
Verus connects Rust code to proof obligations



- Write specifications and proofs alongside executable code.
- Verify supported Rust code with annotations and library specifications.
- Erase specification/proof code when compiling the executable.

Source: Verus project

Source: Verus Guide: code modes



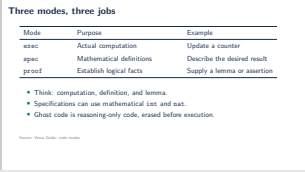
Verus supports a subset of Rust; existing crates are not automatically fully verified. The diagram is conceptual, not a complete description of the internal compiler pipeline.

Three modes, three jobs

Mode	Purpose	Example
`exec`	Actual computation	Update a counter
`spec`	Mathematical definitions	Describe the desired result
`proof`	Establish logical facts	Supply a lemma or assertion

- Think: computation, definition, and lemma.
- Specifications can use mathematical `int` and `nat`.
- Ghost code is reasoning-only code, erased before execution.

Source: Verus Guide: code modes



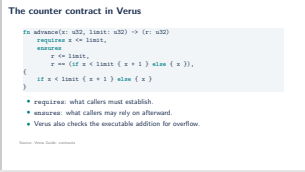
Machine integers still matter in exec code. Mathematical types let the specification describe arithmetic without silently wrapping. Proof functions can package reusable arguments.

The counter contract in Verus

```
fn advance(x: u32, limit: u32) -> (r: u32)
    requires x <= limit,
    ensures
        r <= limit,
        r == (if x < limit { x + 1 } else { x }),
{
    if x < limit { x + 1 } else { x }
}
```

- `requires`: what callers must establish.
- `ensures`: what callers may rely on afterward.
- Verus also checks the executable addition for overflow.

Source: Verus Guide: contracts



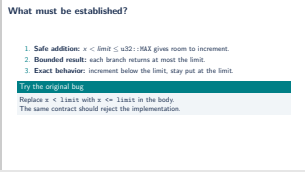
The complete file adds use vstd::prelude::* and a verus! block. In specification expressions, the addition is mathematical. In the executable branch, $x < \text{limit}$ and the u32 bound on limit justify $x + 1$. This is intended to be an automatically discharged example.

What must be established?

1. **Safe addition:** $x < \mathit{limit} \leq$ `u32::MAX` gives room to increment.
2. **Bounded result:** each branch returns at most the limit.
3. **Exact behavior:** increment below the limit, stay put at the limit.

Try the original bug

Replace `x < limit` with `x <= limit` in the body.
The same contract should reject the implementation.



Keep the specification unchanged during this mutation. At equality the body disagrees with the exact postcondition and breaks the bound; at MAX the body can overflow as well. This is a useful demonstration even when not running live.

Using a verified function is applying a lemma

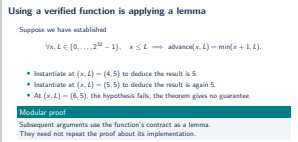
Suppose we have established

$$\forall x, L \in \{0, \ldots, 2^{32} - 1\}, \quad x \leq L \implies \text{advance}(x, L) = \min(x + 1, L).$$

- Instantiate at $(x, L) = (4, 5)$ to deduce the result is 5.
- Instantiate at $(x, L) = (5, 5)$ to deduce the result is again 5.
- At $(x, L) = (6, 5)$, the hypothesis fails; the theorem gives no guarantee.

Modular proof

Subsequent arguments use the function's contract as a lemma.
They need not repeat the proof about its implementation.



The displayed functional equation is justified for this terminating function. General Hoare contracts need the partial-correctness qualification discussed earlier. The corresponding client assertions are retained in examples/counter.rs.

A loop invariant is an induction hypothesis

For `while` B `do` C, choose a predicate I on states.

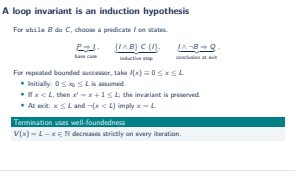
$$\underbrace{P \Rightarrow I}_{\text{base case}}, \qquad \underbrace{\{I \wedge B\}\ C\ \{I\}}_{\text{inductive step}}, \qquad \underbrace{I \wedge \neg B \Rightarrow Q}_{\text{conclusion at exit}}.$$

For repeated bounded successor, take $I(x) \equiv 0 \leq x \leq L$.

- Initially: $0 \leq x_0 \leq L$ is assumed.
- If $x < L$, then $x' = x + 1 \leq L$; the invariant is preserved.
- At exit: $x \leq L$ and $\neg(x < L)$ imply $x = L$.

Termination uses well-foundedness

$V(x) = L - x \in \mathbb{N}$ decreases strictly on every iteration.



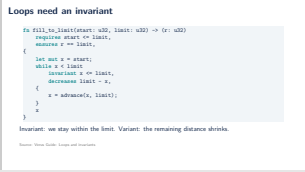
Induct on the number of completed iterations. The invariant includes the state where the guard is false, so x $<$ L would be too strong. Nonnegativity is implicit for u32 in the upcoming code. A strictly descending sequence in the naturals cannot be infinite; this is the separate termination argument.

Loops need an invariant

```
fn fill_to_limit(start: u32, limit: u32) -> (r: u32)
    requires start <= limit,
    ensures r == limit,
{
    let mut x = start;
    while x < limit
        invariant x <= limit,
        decreases limit - x,
    {
        x = advance(x, limit);
    }
    x
}
```

Invariant: we stay within the limit. Variant: the remaining distance shrinks.

Source: Verus Guide: Loops and invariants



Deliberately simple, not a recommended implementation of assignment. Establish the invariant initially, preserve it after each step, then combine it with the negated guard to get equality. The variant is nonnegative and strictly decreases while the guard holds.

What is being trusted?

- The specification captures the intended theorem.
- The translation from code to logical obligations is sound.
- The solver's result and any assumed lemmas are correct.
- Compilation and execution preserve the verified meaning.

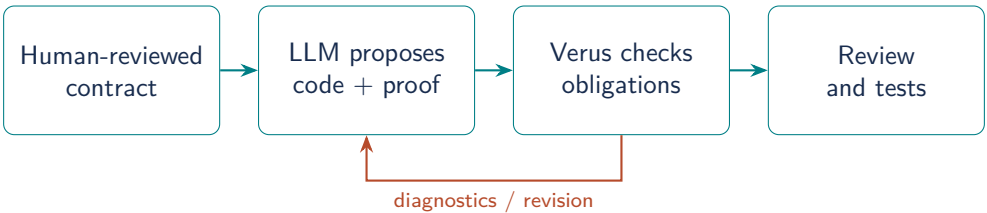
Distinguish proof search from proof checking

Here we rely on the Verus/Z3 verification pipeline.
We are not presenting an independently checked proof certificate.



This is a natural question for an audience familiar with proof assistants. Explain the trust boundary of this workflow without implying that SMT solvers cannot produce proofs or that other proof systems have no trusted components. Our demonstration runs verification; it does not export a certificate to a separate small kernel.

An LLM proposes a construction; verification checks it



Plausibility is not a proof: check the generated construction against a fixed claim.

└An LLM proposes a construction; verification checks it



Plausibility is not a proof: check the generated construction against a fixed claim.

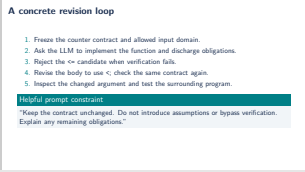
This is a proposed engineering workflow, not a claim that Verus guarantees all LLM output is safe. LLMs can help draft implementations, invariants, and lemmas; the generated proof text itself is not evidence until checked.

A concrete revision loop

1. Freeze the counter contract and allowed input domain.
2. Ask the LLM to implement the function and discharge obligations.
3. Reject the <= candidate when verification fails.
4. Revise the body to use <; check the same contract again.
5. Inspect the changed argument and test the surrounding program.

Helpful prompt constraint

"Keep the contract unchanged. Do not introduce assumptions or bypass verification. Explain any remaining obligations."



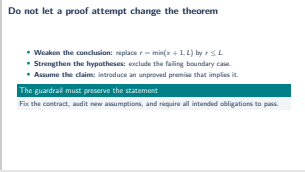
A prompt constraint helps communicate intent but is not enforcement. Repository permissions, review, and CI must enforce the acceptance policy independently. Do not let an agent edit the verifier command to make a failed run appear successful.

Do not let a proof attempt change the theorem

- **Weaken the conclusion:** replace $r = \min(x + 1, L)$ by $r \leq L$.
- **Strengthen the hypotheses:** exclude the failing boundary case.
- **Assume the claim:** introduce an unproved premise that implies it.

The guardrail must preserve the statement

Fix the contract, audit new assumptions, and require all intended obligations to pass.



These are familiar mathematical ways to prove something other than the original claim. An LLM repair loop can make the same moves. Review hypothesis changes even when they remain consistent. Tool errors and timeouts must not be counted as successful verification. Preserve the command, tool versions, and exact submitted code.

The guardrail has a defined reach

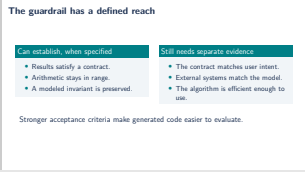
Can establish, when specified

- Results satisfy a contract.
- Arithmetic stays in range.
- A modeled invariant is preserved.

Still needs separate evidence

- The contract matches user intent.
- External systems match the model.
- The algorithm is efficient enough to use.

Stronger acceptance criteria make generated code easier to evaluate.

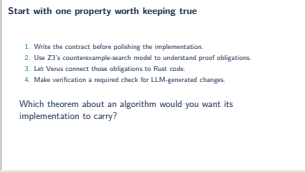


Verification is especially valuable when humans can describe the invariant more reliably than they can inspect every generated implementation. Properties omitted from the contract remain outside the claim.

Start with one property worth keeping true

1. Write the contract before polishing the implementation.
2. Use Z3's counterexample-search model to understand proof obligations.
3. Let Verus connect those obligations to Rust code.
4. Make verification a required check for LLM-generated changes.

Which theorem about an algorithm would you want its implementation to carry?



Invite questions. Possible mathematical examples: a sorting routine preserves the input multiset, Euclid steps preserve the gcd, or a numerical method maintains a certified error bound. Each requires a specification and an appropriate model of arithmetic; floating-point analysis requires additional care.

References and demo files

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus source and installation instructions

  `https://github.com/verus-lang/verus`

- Microsoft's Z3 Guide

  `https://microsoft.github.io/z3guide/`

Local examples: `examples/counter.rs`, `examples/counter-bug.smt2`, `examples/counter-fixed.smt2`.



The README documents build commands and validation status. Source links throughout the deck are clickable. Speaker notes live in the TeX source and are hidden in the audience PDF.